

Chapter 4: Heuristics for the Vehicle Routing Problem

Vehicle Routing: Problems, Methods, and Applications

2nd edition, SIAM 2014

Gilbert Laporte Stefan Ropke Thibaut Vidal

SIAM Monographs on Discrete Mathematics and Applications

Slides prepared for independent study

Table of Contents I

- 1 Introduction
- 2 Constructive Heuristics
- 3 Classical Improvement Heuristics
- 4 Metaheuristics
- 5 Hybridizations
- 6 Unified Algorithms
- 7 Computational Comparison of Selected Metaheuristics
- 8 Conclusions and Future Research Directions
- 9 Summary of Algorithms
- 10 Bibliography

Introduction: Why Heuristics? I

The Vehicle Routing Problem (VRP) in its most basic form asks for a set of minimum-cost routes, all starting and ending at a common *depot*, that collectively visit a given set of customers while respecting vehicle capacity constraints. Because the VRP is NP-hard, exact algorithms can solve instances with only a few hundred customers within reasonable time limits. Real-world logistics instances often contain thousands of customers, making exact methods computationally infeasible.

The Role of Heuristics

A **heuristic** is an algorithm that sacrifices the guarantee of finding an optimal solution in favour of finding a good solution quickly. For the VRP, state-of-the-art heuristics routinely produce solutions within **0.5%** of the best known optimum on standard benchmark instances, at a fraction of the computational cost of exact methods.

The evolution of VRP heuristics spans more than five decades. The first practical heuristic — the Clarke-Wright savings algorithm — was published in 1964. Since the 1990s, **metaheuristics** (Tabu Search, Simulated Annealing, Genetic Algorithms, Ant Colony Optimisation, Large Neighbourhood Search) have superseded classical improvement procedures as the dominant paradigm, and recent **hybrid** and **unified** algorithms combine strengths from several frameworks.

Introduction: Why Heuristics? II

Three Broad Categories

- 1 **Constructive heuristics** build a feasible solution from scratch, inserting customers one at a time or merging routes incrementally. They are fast but produce solutions of moderate quality.
- 2 **Classical improvement heuristics** start from an initial feasible solution and apply local-search moves (edge swaps, customer relocations) to reduce its cost. They refine constructive solutions but may get stuck in local optima.
- 3 **Metaheuristics** guide the search over a much wider neighbourhood, accepting worsening moves (temporarily or probabilistically) to escape local optima. They achieve the best solution quality but require more computation time.

Key Takeaway

Modern practice typically chains these three levels: a constructive heuristic provides an initial solution, local-search operators improve it, and a metaheuristic framework orchestrates the overall search.

The Capacitated VRP: Formal Problem Statement I

Before introducing any algorithm, we need a precise statement of the problem. The **Capacitated Vehicle Routing Problem (CVRP)** is defined on a complete graph $G = (V, A)$, where V is the set of nodes and A is the set of arcs (directed edges) connecting them.

Problem Data (all given as input)

- $V = \{0, 1, 2, \dots, n\}$: the set of nodes. Node 0 is the **depot** (the starting/ending point for all vehicles); nodes 1 through n are the **customers** to be visited.
- d_{ij} (d sub i - j): the travel cost (distance or time) from node i to node j , for every pair $(i, j) \in A$. We assume the triangle inequality holds: $d_{ik} \leq d_{ij} + d_{jk}$.
- q_i (q sub i): the demand (quantity to be delivered) of customer i , for $i = 1, \dots, n$. The depot has $q_0 = 0$.
- Q : the capacity of each vehicle. Every vehicle can carry at most Q units of goods on any single route.
- m : the number of identical vehicles available at the depot.

The Capacitated VRP: Formal Problem Statement II

Decision Variables and Objective

The CVRP seeks to determine a set of at most m routes, each starting and ending at the depot, such that:

- 1 Every customer $i \in \{1, \dots, n\}$ is visited by exactly one route.
- 2 The total demand of customers on each route does not exceed the vehicle capacity Q .
- 3 The total travel cost (sum of d_{ij} over all arcs used) is **minimised**.

In integer-programming notation, let the binary variable $x_{ij} \in \{0, 1\}$ (x sub i-j) equal 1 if some vehicle travels directly from node i to node j , and 0 otherwise. The objective function is then:

$$\text{Minimise } Z = \sum_{i \in V} \sum_{j \in V, j \neq i} d_{ij} x_{ij}$$

where the double $\sum$ (Sigma, meaning “sum”) runs over all *ordered* pairs (i, j) of distinct nodes. This computes the total distance driven by all vehicles across all routes.

The Capacitated VRP: Formal Problem Statement III

How to Read the Objective Function

Z is the total travel distance (or cost) of the entire delivery plan. For each arc $(i \rightarrow j)$ that a vehicle travels, $x_{ij} = 1$ and we add d_{ij} to the sum; for arcs not used, $x_{ij} = 0$ and they contribute nothing. Minimising Z means finding routes that collectively use the shortest possible total path. Adding more vehicles generally reduces each vehicle's individual route length but may be prohibited by the constraint m ; serving more customers per route risks violating capacity Q . The algorithm balances these competing pressures.

Size and Complexity

Even for just $n = 20$ customers and $m = 4$ vehicles, the number of possible route assignments is astronomically large. The CVRP is **NP-hard**: no known algorithm solves all instances in polynomial time. Exact algorithms based on branch-and-bound can handle instances up to roughly $n \approx 200$ in practice; for larger instances, heuristics and metaheuristics are essential.

Constructive Heuristics: Overview I

Constructive heuristics build a feasible solution from scratch without performing any subsequent improvement. Their appeal lies in speed: given n customers they typically run in $O(n^2 \log n)$ time or better. The three most important constructive heuristics for the CVRP are the **Clarke-Wright savings algorithm**, the **nearest-neighbour** heuristic, and the **sweep** algorithm.

Common Starting Point

All three methods begin with a trivial *radial* solution: every customer i is served by a dedicated vehicle that travels depot $\rightarrow i \rightarrow$ depot. The total cost of this solution is $2 \sum_{i=1}^n d_{0i}$, where d_{0i} denotes the distance from the depot (node 0) to customer i . Each method then tries to reduce this cost by consolidating customers into multi-stop routes.

The Clarke-Wright Savings Algorithm (1964) I

The Clarke-Wright savings algorithm, introduced by G. Clarke and J. W. Wright in 1964, is arguably the most influential VRP heuristic ever published. The central idea is simple: if we merge the route depot $\rightarrow i \rightarrow$ depot with the route depot $\rightarrow j \rightarrow$ depot into the single route depot $\rightarrow i \rightarrow j \rightarrow$ depot, we save the two depot-return and depot-departure edges and instead pay for the direct edge between i and j .

Savings Formula

The **saving** from combining customer i and customer j into the same route is defined as:

$$s(i, j) = d_{0i} + d_{0j} - d_{ij}$$

Here, d_{0i} (d sub “zero- i ”) is the distance from the depot (node 0) to customer i ; d_{0j} is the distance from the depot to customer j ; and d_{ij} (d sub “ i - j ”) is the direct travel distance between customer i and customer j . The notation $s(i, j)$ stands for the *saving* achieved by serving i and j on the same route. A positive saving $s(i, j) > 0$ means that linking i and j reduces total travel distance.

The Clarke-Wright Savings Algorithm (1964) II

How to Read This Formula

Think of $d_{0i} + d_{0j}$ as the cost of two separate out-and-back trips from the depot. Subtracting d_{ij} gives the *reduction* in distance when we instead make a single trip depot $\rightarrow i \rightarrow j \rightarrow$ depot (or depot $\rightarrow j \rightarrow i \rightarrow$ depot). The larger the saving, the more we gain by linking these two customers together. If the two customers are far from each other but both far from the depot (large d_{0i} and d_{0j}), the saving is large. If they are close to each other (small d_{ij}), the saving is also large.

The algorithm greedily merges routes in order of decreasing savings, subject to two feasibility conditions: (1) the merge must not violate vehicle capacity, and (2) i and j must currently be at the *interior endpoints* of their respective routes (i.e., the customers that are directly adjacent to the depot).

The Clarke-Wright Savings Algorithm (1964) III

Clarke-Wright Parallel Algorithm

- ① Compute saving $s(i, j)$ for every pair (i, j) with $i \neq j$.
- ② Sort all pairs in **decreasing** order of $s(i, j)$.
- ③ Start with n single-customer routes (depot $\rightarrow i \rightarrow$ depot for each i).
- ④ For each pair (i, j) in sorted order, **merge** the route ending at i with the route beginning at j into a single route, provided:
 - i and j belong to *different* current routes,
 - i is the last customer before the depot in its route,
 - j is the first customer after the depot in its route,
 - The combined load does not exceed the vehicle capacity Q .
- ⑤ Repeat until no more merges are feasible.

The Clarke-Wright Savings Algorithm (1964) IV

Worked Example: 4 Customers, $Q = 10$

Suppose the depot is at origin $(0,0)$ and four customers are located at $c_1 = (3,0)$, $c_2 = (5,0)$, $c_3 = (0,3)$, $c_4 = (0,5)$ with demands $q_1 = q_2 = q_3 = q_4 = 3$ and Euclidean distances. Then:

$$s(1,2) = 3 + 5 - 2 = 6 \quad (\text{very profitable: both on same axis})$$

$$s(3,4) = 3 + 5 - 2 = 6$$

$$s(1,3) = 3 + 3 - \sqrt{18} \approx 1.76$$

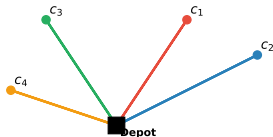
$$s(2,4) = 5 + 5 - \sqrt{50} \approx 2.93$$

Merging $(1,2)$ first gives route $D \rightarrow 1 \rightarrow 2 \rightarrow D$ (load = $6 \leq 10$). Merging $(3,4)$ gives route $D \rightarrow 3 \rightarrow 4 \rightarrow D$ (load = $6 \leq 10$). Final solution: **two routes**, total distance = $2 + 2 + 2 + 2 + 3 + 5 = 16$ vs. initial $2(3 + 5 + 3 + 5) = 32$. Savings = 50%.

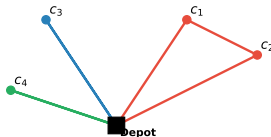
The Clarke-Wright Savings Algorithm (1964) V

Clarke-Wright Savings Algorithm — Step-by-Step Merging

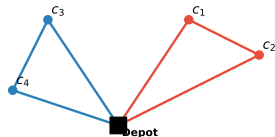
Step 1: All routes via depot



Step 2: Merge best saving (1,2)



Step 3: Merge (3,4)



Key Insight

The Clarke-Wright algorithm is greedy and does not backtrack. It runs in $O(n^2 \log n)$ time (dominated by the sort of savings) and typically produces solutions **10–15%** above the optimum. A *sequential* variant processes savings one by one and extends a single active route; the *parallel* variant (shown above) builds all routes simultaneously and generally gives better results.

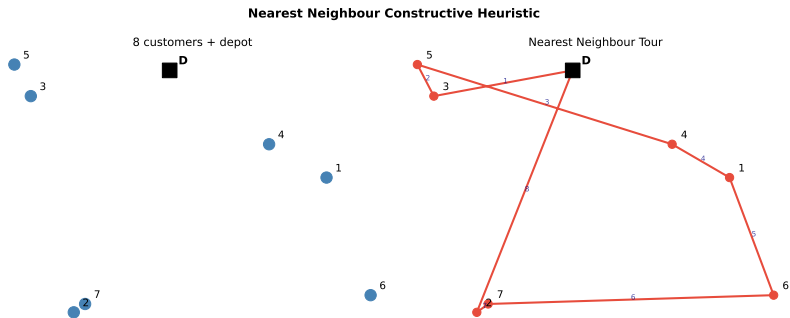
Nearest Neighbour Heuristic I

The nearest-neighbour (NN) heuristic is one of the simplest constructive methods for the Travelling Salesman Problem and is easily adapted to the VRP. The idea is to mimic how a driver naturally plans a delivery tour: at each step, visit the *closest unvisited customer* that can be reached without violating the vehicle's capacity constraint.

Nearest Neighbour Algorithm for CVRP

- ① Set current position $\leftarrow$ depot; remaining capacity $\leftarrow Q$; all customers unvisited.
- ② While unvisited customers remain:
 - (a) Among all unvisited customers j with demand $q_j \leq$ remaining capacity, select the one with minimum distance $d(\text{current}, j)$.
 - (b) If no such customer exists, return to the depot and start a new route (reset remaining capacity to Q).
 - (c) Otherwise, append j to the current route; update current position and remaining capacity.
- ③ Return all vehicles to the depot.

Nearest Neighbour Heuristic II



Nearest Neighbour Heuristic III

Worked Example: 5 Customers

Customers at positions: $c_1 = (1, 1)$, $c_2 = (3, 2)$, $c_3 = (2, 4)$, $c_4 = (5, 1)$, $c_5 = (4, 3)$; depot at $(0, 0)$; all demands $q_i = 2$; $Q = 6$ (3 customers per route).

Route 1 starts at depot $(0, 0)$. Nearest unvisited: c_1 at distance $\sqrt{2} \approx 1.41$. From c_1 : nearest is c_2 at distance $\sqrt{(2^2 + 1^2)} = \sqrt{5} \approx 2.24$. Remaining capacity after c_1, c_2 : $6 - 4 = 2$, so one more customer fits. From c_2 : nearest with demand ≤ 2 is c_5 at $\sqrt{(1^2 + 1^2)} = \sqrt{2}$. Route full. Return to depot.

Route 2: nearest of $\{c_3, c_4\}$ from depot is c_4 at $\sqrt{26} \approx 5.1$. From c_4 : c_3 is at $\sqrt{9 + 9} \approx 4.24$. Route full. Return to depot.

Final solution: Route 1: $D \rightarrow c_1 \rightarrow c_2 \rightarrow c_5 \rightarrow D$; Route 2: $D \rightarrow c_4 \rightarrow c_3 \rightarrow D$.

Nearest Neighbour Heuristic IV

Key Insight

The nearest-neighbour heuristic is very fast ($O(n^2)$) but produces solutions that can be 20–25% above optimal, because greedily choosing the nearest neighbour early on may create expensive return trips later. It is best used as a starting point for improvement procedures.

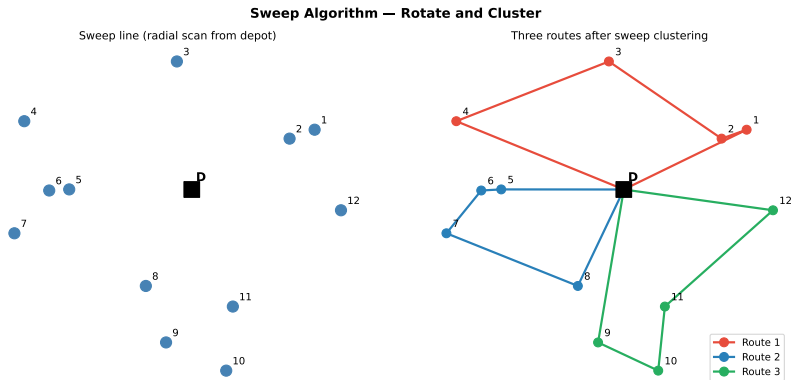
Sweep Algorithm (Gillet & Miller, 1974) I

The sweep algorithm, introduced by Gillet and Miller in 1974, exploits the geometric structure of the VRP. It converts the 2-D routing problem into a much simpler 1-D packing problem by representing each customer's position in polar coordinates centred at the depot. Customers are then clustered by *sweeping* a ray from the depot in a fixed direction and grouping consecutive customers until the vehicle capacity is exhausted.

Sweep Algorithm

- 1 Convert each customer i to polar coordinates (ρ_i, θ_i) relative to the depot, where ρ_i is the distance and θ_i is the angle.
- 2 Sort customers by angle θ_i (breaking ties by distance).
- 3 Rotate a sweep line counterclockwise starting from angle $\theta_{\min}$. Assign customers to the current vehicle in angular order until the capacity Q would be exceeded; then close that route and start a new vehicle.
- 4 Within each cluster, solve the single-vehicle TSP (or use a nearest-neighbour sub-tour) to determine the visit order.

Sweep Algorithm (Gillet & Miller, 1974) II



Sweep Algorithm (Gillet & Miller, 1974) III

Worked Example

Suppose 9 customers are sorted by angle: $\theta_1 < \theta_2 < \dots < \theta_9$, with demands all equal to 3 and $Q = 9$. The sweep groups customers $\{1, 2, 3\}$ into Route 1 (total demand = 9, exactly at capacity), $\{4, 5, 6\}$ into Route 2, and $\{7, 8, 9\}$ into Route 3. Each route's visit order is then determined by solving the small TSP on its 3 customers.

Key Insight

The sweep algorithm is extremely fast and naturally produces compact, geographically clustered routes. Its main weakness is that the quality of the cluster-first, route-second decomposition depends on the starting angle; running the sweep from multiple starting angles and keeping the best result significantly improves solution quality.

Classical Improvement Heuristics: Overview I

Classical improvement heuristics (also called *local search* methods) start from a feasible solution and iteratively apply *moves* that replace the current solution with a better neighbour. A **move** is a small, reversible change to the solution (e.g., swapping two edges, relocating one customer). The algorithm terminates at a **local optimum**: a solution from which no single move produces an improvement.

General Local-Search Framework

Input: initial feasible solution s_0

Repeat:

- Enumerate all moves in the neighbourhood $\mathcal{N}(s)$.
- If a move m exists such that $c(s \oplus m) < c(s)$, apply the *best* (or first) improving move: $s \leftarrow s \oplus m$.

Until no improving move exists.

Return s .

The key design choices are the **neighbourhood structure** (which moves are considered) and the **pivot rule** (best-improving vs. first-improving). Common VRP neighbourhood operators are 2-opt, 3-opt, Or-opt, and the Lin-Kernighan move.

2-opt Local Search I

The 2-opt move was introduced by Lin (1965) for the TSP and is the most widely used improvement operator. The intuition is that an optimal tour should never have two *crossing* edges, because uncrossing them always reduces the total distance. A 2-opt move removes two edges from the current tour and reconnects the two resulting path segments in the only other feasible way.

2-opt Local Search II

2-opt Move Definition

Given a route $R = (v_0, v_1, \dots, v_n, v_0)$ and two indices $1 \leq i < j \leq n$, the 2-opt move **removes** edges (v_{i-1}, v_i) and (v_j, v_{j+1}) and **adds** edges (v_{i-1}, v_j) and (v_i, v_{j+1}) , which is equivalent to **reversing** the segment $v_i, v_{i+1}, \dots, v_j$.

The **improvement** from this move is defined as:

$$\Delta \text{ (delta, meaning "change")} = d(v_{i-1}, v_i) + d(v_j, v_{j+1}) - d(v_{i-1}, v_j) - d(v_i, v_{j+1})$$

Here Δ (the Greek letter *Delta*) represents the net reduction in travel distance from making this swap. The terms $d(v_{i-1}, v_i)$ and $d(v_j, v_{j+1})$ are the lengths of the two edges being *removed*; the terms $d(v_{i-1}, v_j)$ and $d(v_i, v_{j+1})$ are the lengths of the two edges being *added*. The move is applied whenever $\Delta > 0$, meaning the new edges are collectively shorter than the ones they replace.

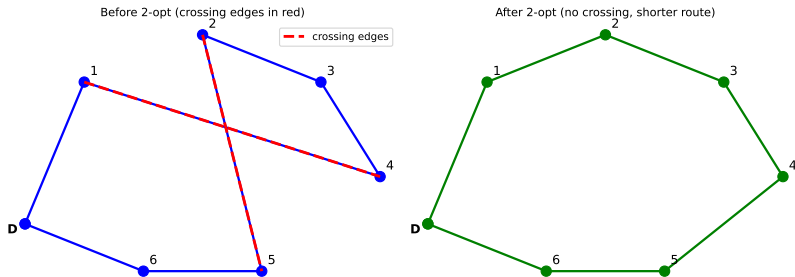
2-opt Local Search III

How to Read This Formula

Δ is the *distance saved* by removing the two old edges and adding the two new ones. If the two edges being removed “cross” in the plane, then the triangle inequality guarantees $\Delta > 0$: uncrossing two crossing edges always shortens the tour. When Δ is large, the improvement is dramatic; when it is near zero, the move is marginal. The algorithm keeps applying 2-opt moves as long as any $\Delta > 0$ can be found.

2-opt Local Search IV

2-opt Move: Remove Two Edges and Reconnect



2-opt Local Search V

Worked Example: 2-opt Improvement

Consider a single-route solution visiting customers in order 1, 3, 5, 2, 4 with coordinates $c_1 = (0, 0)$, $c_2 = (2, 0)$, $c_3 = (1, 2)$, $c_4 = (3, 2)$, $c_5 = (2, 4)$ and depot at $(0, 0)$. The current route $D \rightarrow 1 \rightarrow 3 \rightarrow 5 \rightarrow 2 \rightarrow 4 \rightarrow D$ has a crossing between edges $(1, 3) \rightarrow (5, 2)$ and the path between c_3 and c_2 . A 2-opt move on indices $i = 2, j = 4$ reverses the segment 3, 5, 2 to 2, 5, 3, giving route $D \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 3 \rightarrow 4 \rightarrow D$, which removes the crossing and reduces the total distance.

Key Insight

The 2-opt neighbourhood has size $O(n^2)$. A complete 2-opt pass over a route of length n runs in $O(n^2)$ time. The 2-opt local optimum is typically 5–10% above the global optimum for random Euclidean instances. For the VRP, 2-opt can also operate *between* routes (intra-route and inter-route 2-opt).

3-opt and Lin-Kernighan Moves I

The 3-opt move generalises 2-opt by removing *three* edges and reconnecting the three resulting segments in the best possible way. There are 8 ways to reconnect 3 segments (including the original), and 3-opt considers all of them. The Lin-Kernighan (LK) heuristic (1973) goes further and applies a *variable-depth* search: it dynamically decides how many edges to exchange at each step, effectively exploring moves of order $k = 2, 3, 4, \dots$ in a single sequential procedure.

3-opt: Three Edge Removal and Reconnection

Given a tour and three edge indices, remove edges (v_{i-1}, v_i) , (v_{j-1}, v_j) , (v_{k-1}, v_k) . The three resulting path segments can be reconnected in multiple ways. The improvement Δ is computed for each reconnection, and the best improving reconnection is applied.

3-opt and Lin-Kernighan Moves II

Lin-Kernighan Variable-Depth Search

The LK algorithm starts by removing edge (t_1, t_2) and sequentially adds a new edge (t_2, t_3) , removes the edge on the tour from t_3 , adds the next edge, and so on, building a “chain” of improving candidate edges. At each step it checks whether closing the chain (reconnecting the tour) gives an overall improvement. The search terminates when no extension leads to improvement. This sequential structure allows LK to efficiently discover many high-order moves that simpler k -opt methods would miss.

Key Insight

The LK heuristic for the TSP is one of the most powerful classical improvement methods, producing solutions typically within 1–2% of optimal. Its adaptation to the VRP (as part of LKH-style solvers) is more complex because feasibility (capacity) constraints must be maintained throughout the chain construction.

Or-opt: Customer Relocation I

Or-opt moves, introduced by Or (1976), are a family of local-search operators that relocate a contiguous *segment* of one or more customers from their current route to a different position (within the same route or in a different route). Unlike 2-opt which reverses a segment, Or-opt keeps the segment's internal order intact.

Or-opt: Customer Relocation II

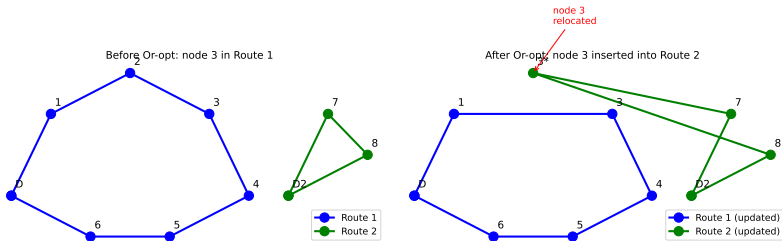
Or-opt Move Types

- **Or-opt-1** (single relocation): Remove one customer v from its current position and insert it elsewhere. Let u and w denote the *predecessor* (node just before v in the route) and *successor* (node just after v) respectively. Removing v saves the two edges $d(u, v)$ and $d(v, w)$ but adds the shortcut edge $d(u, w)$, giving a **removal gain** of $\delta_{\text{rem}} = d(u, v) + d(v, w) - d(u, w)$. Inserting v between nodes a and b at a new position costs $\delta_{\text{ins}} = d(a, v) + d(v, b) - d(a, b)$. The net gain of the entire Or-opt-1 move is $\Delta = \delta_{\text{rem}} - \delta_{\text{ins}}$. The move is applied when $\Delta > 0$.
- **Or-opt-2**: Relocate a sequence of two consecutive customers (v_1, v_2) as a unit, keeping their internal order.
- **Or-opt-3**: Relocate a sequence of three consecutive customers as a unit.

In all cases, the new position must respect the vehicle capacity Q .

Or-opt: Customer Relocation III

Or-opt Move: Relocate One (or Two) Customers Between Routes



Or-opt: Customer Relocation IV

Worked Example: Or-opt-1

Route 1: $D \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow D$ with loads (3, 3, 4), total load = 10 = Q . Route 2: $D \rightarrow 4 \rightarrow 5 \rightarrow D$ with loads (2, 2), total load = 4. Consider relocating customer 3 (load 4) to Route 2: new load of Route 2 = $4 + 4 = 8 \leq Q = 10$. The new routes are $D \rightarrow 1 \rightarrow 2 \rightarrow D$ and $D \rightarrow 4 \rightarrow 3 \rightarrow 5 \rightarrow D$. If $d(2, 3) + d(3, D) - d(2, D) > d(4, 3) + d(3, 5) - d(4, 5)$ (removal saving exceeds insertion cost), the move reduces total distance.

Key Insight

Or-opt-1 is particularly effective for VRP because it can transfer customers between routes, reducing the total number of vehicles used and balancing route loads. The Or-opt neighbourhood is strictly contained in the 3-opt neighbourhood. In practice, combining 2-opt and Or-opt-1 within a single local-search procedure gives significantly better results than either operator alone.

Metaheuristics: Escaping Local Optima I

Classical local search terminates at a local optimum — a solution that no single move improves — but this local optimum may be far from the global optimum. **Metaheuristics** are higher-level strategies that guide local search to escape local optima and explore the solution space more broadly.

Three Paradigms for Escaping Local Optima

- 1 **Memory-based** (Tabu Search): Forbid recently used moves to prevent cycling and force exploration of new regions.
- 2 **Probabilistic acceptance** (Simulated Annealing): Accept worsening moves with a probability that decreases over time.
- 3 **Population-based** (Genetic Algorithms, ACO): Maintain a set of solutions and combine them to generate new ones.

Since the publication of the first edition of the VRP book (2002), metaheuristics have become the dominant algorithmic paradigm for the VRP, with modern implementations consistently delivering solutions within **0.1–0.5%** of the best known values on standard benchmarks.

Tabu Search I

The word *tabu* (also spelled *taboo*) means “forbidden.” Tabu Search (Glover, 1986) maintains a **tabu list** — a short-term memory of recently applied moves. At each iteration, the algorithm selects the best non-tabu move, even if it worsens the objective. This prevents cycling back to recently visited solutions and forces the search into unexplored parts of the solution space.

Tabu Search II

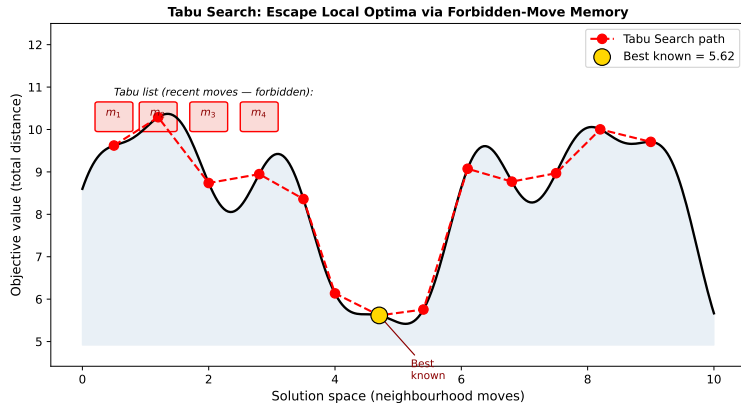
Tabu Search Framework

Given: initial solution s ; tabu list $\mathcal{T}$ (calligraphic T) = $\emptyset$ (empty set); best known solution $s^* = s$; tabu tenure τ (tau, pronounced “taw”, a positive integer giving the number of iterations a move remains forbidden).

- 1 Generate all neighbours $\mathcal{N}(s)$ (the neighbourhood of s).
- 2 Select the best $s' \in \mathcal{N}(s)$ such that the move $m(s \rightarrow s')$ is *not* in $\mathcal{T}$ **or** s' satisfies the **aspiration criterion** (i.e., $c(s') < c(s^*)$, meaning it is better than the best known solution regardless of tabu status).
- 3 Set $s \leftarrow s'$; add move m to $\mathcal{T}$; if $|\mathcal{T}| > \tau$ (the list exceeds the tenure), remove the oldest entry.
- 4 If $c(s) < c(s^*)$, update $s^* \leftarrow s$.
- 5 Repeat until stopping criterion (time limit or iteration limit) is reached.

Return s^* (the best solution found throughout the entire run).

Tabu Search III



Tabu Search for VRP: Key Design Choices

- **Move type:** Or-opt relocations (remove customer v from route R_1 , insert into route R_2) are the most common tabu move for VRP.
- **What is made tabu:** The pair (v, R_1) is placed on the tabu list, preventing customer v from returning to route R_1 for the next τ iterations.
- **Tabu tenure τ :** The length of the tabu list. Typical values for VRP are $\tau \in [5, 20]$. Too short a tenure leads to cycling; too long a tenure over-restricts the search.
- **Penalty for infeasibility:** Many VRP Tabu Search implementations allow temporarily infeasible solutions (exceeding capacity) and penalise them with a term $\lambda \cdot \max(0, \text{excess load})$, where λ is a self-adjusting penalty coefficient. This enlarges the effective search space.

Among the most successful Tabu Search algorithms for the VRP are those of Taillard (1993), Rochat and Taillard (1995), Gendreau, Hertz, and Laporte (1994) (GENIUS/TABUROUTE), and the implementations of Cordeau, Gendreau, and Laporte (1997) for the VRPTW.

Key Insight

The aspiration criterion is critical: it allows the algorithm to accept a tabu move when doing so would produce a new best solution. Without it, the tabu list might prevent the discovery of the global optimum.

Simulated Annealing I

The analogy behind Simulated Annealing (SA) comes from metallurgy: when a metal is heated to a high temperature and then slowly cooled (*annealed*), its atoms settle into a low-energy crystalline structure. Kirkpatrick, Gelatt, and Vecchi (1983) observed that this physical process can be mimicked in combinatorial optimisation to escape local optima.

Simulated Annealing II

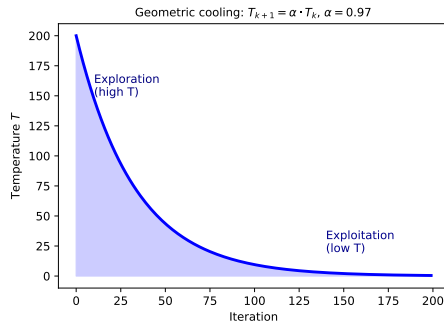
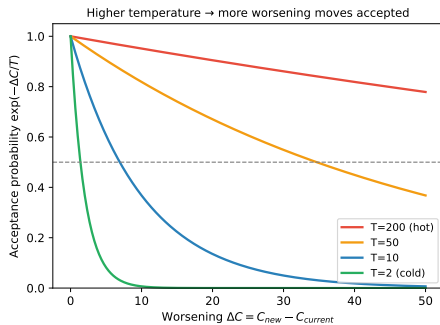
Simulated Annealing Algorithm

Given: initial solution s , initial temperature T_0 , cooling rate $\alpha \in (0, 1)$, stopping temperature $T_{\min}$.

- ① Set $T \leftarrow T_0$, $s^* \leftarrow s$.
- ② **While** $T > T_{\min}$:
 - (a) Sample a random neighbour $s' \in \mathcal{N}(s)$.
 - (b) Compute $\Delta C = c(s') - c(s)$.
 - (c) If $\Delta C < 0$: accept $s \leftarrow s'$ (improvement always accepted).
 - (d) If $\Delta C \geq 0$: accept $s \leftarrow s'$ with probability $p = \exp(-\Delta C/T)$.
 - (e) If $c(s) < c(s^*)$: update $s^* \leftarrow s$.
- ③ Apply cooling: $T \leftarrow \alpha \cdot T$.
- ④ **Return** s^* .

Simulated Annealing III

Simulated Annealing — Temperature Controls Solution Acceptance



Simulated Annealing IV

Interpreting the Acceptance Probability

The acceptance probability formula is $p = \exp(-\Delta C / T)$, where $\exp$ denotes the mathematical constant $e \approx 2.718$ raised to the power $(-\Delta C / T)$. Here, ΔC (delta C) is the cost *increase* of the worsening move (a positive number), and T (the temperature) is a positive control parameter that decreases over time. This probability has two important properties. First, it is always between 0 and 1, so the algorithm may accept worsening moves but never with certainty. Second, as T decreases toward zero, $p \rightarrow 0$ for any $\Delta C > 0$, meaning the algorithm becomes increasingly deterministic and converges toward a pure local search. The geometric cooling schedule $T_k = \alpha^k T_0$ (where α (alpha) $\in (0, 1)$ is the cooling rate, typically 0.95–0.99) is the most common in practice.

Simulated Annealing V

How to Read the Acceptance Formula

The fraction $\Delta C/T$ is the ratio of “how bad the move is” to “how hot the system currently is.” When T is large (hot phase, early in the run), this ratio is small, so $\exp(-\text{small}) \approx 1$ and almost any move is accepted — the algorithm explores broadly. When T is small (cold phase, late in the run), this ratio is large, so $\exp(-\text{large}) \approx 0$ and only improving moves are accepted — the algorithm converges. This mirrors the physical process of annealing: a hot metal can be reshaped easily, but once cooled it holds its structure rigidly.

Numerical Example of Acceptance

Suppose the current solution has cost $c(s) = 1000$ and a randomly sampled neighbour has cost $c(s') = 1020$, so $\Delta C = 20$. At temperature $T = 50$: acceptance probability = $\exp(-20/50) \approx 0.67$ (67% chance of accepting the worse solution). At $T = 5$: probability = $\exp(-20/5) \approx 0.018$ (less than 2%). This shows how early in the run (high T) the algorithm explores broadly, while late in the run (low T) it exploits a good region.

Key Insight

SA's strength is its theoretical guarantee of convergence to the global optimum under a sufficiently slow cooling schedule (*logarithmic cooling*), although in practice geometric cooling is used for computational efficiency. SA applied to the VRP with Or-opt moves was popularised by Osman (1993) and Thompson and Psaraftis (1993).

Genetic Algorithms I

Genetic Algorithms (GAs), introduced by Holland (1975) and applied to combinatorial optimisation by Goldberg (1989), are population-based metaheuristics inspired by biological evolution. The key idea is to maintain a **population** of solutions (“individuals”) and iteratively combine pairs of solutions via **crossover** and **mutation** to produce new solutions (“offspring”). Good solutions survive preferentially to the next generation through **selection**.

Genetic Algorithm for VRP

- ➊ **Initialise** a population of P feasible VRP solutions (using constructive heuristics or random construction).
- ➋ **Evaluate** each solution's fitness $= 1/c(s)$.
- ➌ **While** stopping criterion not met:
 - (a) **Selection:** Choose two parents s_1, s_2 by tournament or roulette-wheel selection.
 - (b) **Crossover:** Apply an order-preserving crossover operator (e.g., OX or PMX) to produce child s' .
 - (c) **Repair:** Fix infeasibilities (duplicate customers, capacity violations).
 - (d) **Mutation:** With probability p_m , apply a random local move to s' .
 - (e) **Local search:** Optionally improve s' with 2-opt or Or-opt (creating a “memetic algorithm”).
 - (f) **Replacement:** Insert s' into the population, removing the worst individual.
- ➍ **Return** best solution found.

Genetic Algorithms III

Genetic Algorithm: Order Crossover (OX) on VRP Chromosomes

Parent 1:

[D→1→2→3→D] [D→4→5→D]

Parent 2:

[D→3→1→4→D] [D→2→5→D]

↓ OX Crossover

Child 1:

[D→1→2→3→D] [D→4→5→D] (inherits P1 segment, fixes P2 order)

Child 2:

[D→3→1→4→D] [D→2→5→D] (inherits P2 segment, fixes P1 order)



Repair infeasible routes (capacity check) → Mutation (swap 2 customers) → Next gen

VRP Crossover: The Challenge of Feasibility

Unlike binary or integer problems, VRP solutions are permutations of customers partitioned into routes. Standard crossover operators can produce offspring with duplicate customers (a customer visited by two routes) or missing customers. The **Order Crossover** (OX) operator handles this by inheriting one parent's route structure for a subset of customers and filling in the rest in the order they appear in the second parent. Capacity constraints must then be re-checked and routes split or merged to restore feasibility.

Among the most successful GAs for the VRP are the Hybrid Genetic Algorithm (HGA) of Vidal et al. (2012, 2014), which combines an advanced OX-based crossover with intensive local search (2-opt, Or-opt, 3-opt) and a diversity management mechanism that prevents premature convergence. The HGA has achieved top performance on virtually all standard VRP benchmarks.

Key Insight

The term **memetic algorithm** describes a GA that includes local search as a post-crossover improvement step. For the VRP, the local-search component is essential: without it, GAs typically produce solutions of similar quality to constructive heuristics. The power of GAs lies in their ability to combine *building blocks* from different good solutions via crossover.

Ant Colony Optimisation (ACO) I

Ant Colony Optimisation was introduced by Dorigo, Maniezzo, and Colorni (1991) inspired by the foraging behaviour of real ants. Real ants deposit **pheromone** on paths they travel, and other ants preferentially follow high-pheromone paths. Over time, shorter paths accumulate more pheromone (because ants return faster and reinforce them more) and longer paths evaporate their pheromone. ACO mimics this collective behaviour to construct good solutions.

Ant Colony Optimisation (ACO) II

ACO Algorithm for VRP

Initialise: pheromone $\tau(i, j) = \tau_0$ for all edges; heuristic desirability $\eta(i, j) = 1/d_{ij}$.

① **For** each of m ants:

- (a) Construct a complete VRP solution by building routes probabilistically: at position i , ant k chooses the next customer j with probability

$$p_{ij}^k = \frac{[\tau(i, j)]^\alpha \cdot [\eta(i, j)]^\beta}{\sum_{\ell \in \mathcal{F}} [\tau(i, \ell)]^\alpha \cdot [\eta(i, \ell)]^\beta}$$

In this formula: $\tau(i, j)$ (tau, pronounced “taw”) is the pheromone level on the edge from customer i to customer j — a number that increases when ants use that edge in good solutions; $\eta(i, j)$ (eta, pronounced “ay-ta”) is the heuristic desirability, usually set to $1/d_{ij}$ (the reciprocal of the distance, so nearer customers are more desirable); α (alpha) is a positive parameter controlling how strongly pheromone influences the choice; β (beta) is a positive parameter controlling how strongly distance influences the choice; $\mathcal{F}$ is the set of feasible next customers (those whose demand fits within the remaining vehicle capacity); and ℓ (ell) is a dummy index summing over all feasible candidates. The denominator normalises the result so that p_{ij}^k is a true probability summing to 1 over all $j \in \mathcal{F}$.

- (b) Optionally improve each ant’s solution with local search.

Large Neighbourhood Search (LNS) I

Large Neighbourhood Search (LNS), introduced by Shaw (1998) for the VRP, is based on the principle that good solutions share structural features (routes, sub-sequences) with other good solutions. Rather than making small local moves, LNS alternates between a **destroy** phase (removing a large portion of the current solution) and a **repair** phase (rebuilding the destroyed portion, ideally better than before).

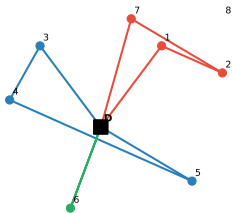
LNS Algorithm

- 1 **Initialise:** construct a feasible solution s ; set $s^* \leftarrow s$.
- 2 **Repeat** until stopping criterion:
 - (a) **Destroy:** Remove q customers from s using a destroy operator (random removal, worst removal, or Shaw's related removal). Obtain partial solution s_{partial} .
 - (b) **Repair:** Re-insert all removed customers into s_{partial} using a greedy or regret-based insertion heuristic. Obtain s' .
 - (c) **Accept:** If $c(s') < c(s)$, set $s \leftarrow s'$. (Simulated Annealing acceptance can also be used.)
 - (d) If $c(s') < c(s^*)$, set $s^* \leftarrow s'$.
- 3 **Return** s^* .

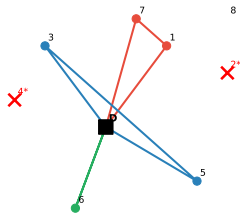
Large Neighbourhood Search (LNS) II

Large Neighbourhood Search (LNS) — Destroy and Repair

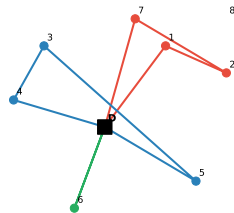
Current solution



Destroy: remove cust. 2,4



Repair: reinsert optimally



Large Neighbourhood Search (LNS) III

Shaw's Related Removal Operator

Shaw's destroy operator removes customers that are *related* to a seed customer u : a customer v is related to u if it is geographically close, has a similar time window, or is on the same route. The relatedness of customers u and v is measured as:

$$\text{rel}(u, v) = \phi \cdot d_{uv} + \chi \cdot |T_u - T_v| + \psi \cdot (1 - \mathbf{1}[\text{same route}])$$

In this formula, ϕ (phi, pronounced “fye”) weights geographical proximity; d_{uv} (d sub u-v) is the distance between customers u and v ; χ (chi, pronounced “kye”) weights time-window similarity; $|T_u - T_v|$ is the absolute difference between the earliest service times of u and v ; ψ (psi, pronounced “sigh”) weights route co-membership; and $\mathbf{1}[\text{same route}]$ equals 1 if u and v are on the same route, 0 otherwise. A *small* value of $\text{rel}(u, v)$ means the customers are highly related. Removing related customers creates a “hole” in the solution that can be repaired in multiple ways, enabling large improvements.

Large Neighbourhood Search (LNS) IV

How to Read the Relatedness Formula

Think of $\text{rel}(u, v)$ as a “dissimilarity score.” Customers that are close together geographically (d_{uv} small), have similar delivery time requirements ($|T_u - T_v|$ small), and are currently on the same route all score low — meaning they are highly related. Removing a cluster of highly related customers leaves a coherent geographical region unserved, and the repair operator can fill that region in a completely different way, potentially discovering a much better solution. By contrast, removing random unrelated customers creates scattered gaps that are easier for a greedy repair to fill but harder to restructure into something significantly better.

Adaptive LNS (ALNS) — Pisinger and Ropke (2007)

The Adaptive LNS extends LNS by maintaining a *bank* of destroy and repair operators and selecting them probabilistically, with probabilities updated based on past performance (operators that have recently improved the solution are chosen more often). ALNS is one of the most versatile and successful metaheuristics for VRP, with implementations achieving top results on the CVRP, VRPTW, PDPTW, and many other variants.

Large Neighbourhood Search (LNS) V

Key Insight

LNS operates on a much larger neighbourhood than classical local search: removing and reinserting q customers from n total creates a neighbourhood of size $\binom{n}{q}$, which is exponential in q . Yet each iteration remains computationally tractable because the repair phase uses efficient insertion heuristics rather than exhaustive enumeration.

Python Implementation: Clarke-Wright Savings Algorithm

```
import numpy as np

def clarke_wright_savings(coords, demands, capacity):
    """
    Clarke-Wright parallel savings algorithm for CVRP.
    coords : list of (x,y); index 0 is the depot
    demands : list of demands; demands[0] = 0 (depot)
    capacity: vehicle capacity Q
    Returns a list of routes (each route is a list of customer indices).
    """
    n = len(coords) - 1 # number of customers (1..n)
    # Euclidean distance matrix
    def dist(i, j):
        return np.hypot(coords[i][0]-coords[j][0], coords[i][1]-coords[j][1])

    # Step 1: compute all savings s(i,j) for i,j in 1..n
    savings = []
    for i in range(1, n+1):
        for j in range(i+1, n+1):
            s = dist(0, i) + dist(0, j) - dist(i, j)
            savings.append((s, i, j))
    savings.sort(reverse=True) # sort descending

    # Step 2: initialise n single-customer routes
    routes = {i: [i] for i in range(1, n+1)} # route_id -> [customers]
    in_route = {i: i for i in range(1, n+1)} # customer -> route_id
    loads = {i: demands[i] for i in range(1, n+1)}
```

Python Implementation: 2-opt Local Search

```
import numpy as np

def two_opt(route, dist_matrix):
    """
    Apply 2-opt improvement to a single TSP route.
    route      : list of node indices (includes depot at start/end)
    dist_matrix: 2D numpy array of distances
    Returns improved route.
    """
    best = route[:]
    improved = True
    while improved:
        improved = False
        n = len(best)
        for i in range(1, n - 2):          # i is first edge start
            for j in range(i + 1, n - 1):  # j is second edge start
                # Current edges: (best[i-1], best[i]) and (best[j], best[j+1])
                d_before = (dist_matrix[best[i-1], best[i]]
                           + dist_matrix[best[j], best[j+1]])
                # After reversing segment best[i..j]:
                d_after = (dist_matrix[best[i-1], best[j]]
                          + dist_matrix[best[i], best[j+1]])
                if d_after < d_before - 1e-10:
                    best[i:j+1] = best[i:j+1][::-1]  # reverse segment
                    improved = True
    return best
```

Python Implementation: Or-opt-1 (Customer Relocation)

```
import numpy as np
from itertools import product

def or_opt1(routes, coords, demands, capacity):
    """
    Or-opt-1: try relocating every customer to every other position
    in any route. Apply the best improving move and repeat.
    """
    n_nodes = len(coords)
    D = np.zeros((n_nodes, n_nodes))
    for i in range(n_nodes):
        for j in range(n_nodes):
            D[i, j] = np.hypot(coords[i][0]-coords[j][0],
                               coords[i][1]-coords[j][1])

    def route_load(r):
        return sum(demands[c] for c in r)

    def route_cost(r):
        full = [0] + r + [0]
        return sum(D[full[k], full[k+1]] for k in range(len(full)-1))

    improved = True
    while improved:
        improved = False
        best_gain, best_move = 0, None
        for ri in range(len(routes)):
            for rj in range(len(routes)):
                if ri == rj:
                    continue
                # Move customer from route ri to route rj
                # (The actual move logic is omitted here for brevity,
                # but it would involve removing a customer from route ri
                # and inserting it into route rj, then checking if the
                # total cost is reduced.)
                # ...
            
```

Hybridizations I

A **hybridization** of metaheuristics combines ideas from two or more paradigms in order to benefit from the strengths of each. Population-based methods such as genetic algorithms are good at *diversification* (exploring many different regions of the solution space) but slow at *intensification* (refining a particular good solution). Conversely, single-solution methods such as Tabu Search excel at intensification but may miss important regions of the solution space.

Main Hybridization Paradigms for VRP

- ① **Population-based search + local search (Memetic Algorithms):** The most successful approach. Each individual in the population is locally optimised (using 2-opt, Or-opt, LK) before being evaluated. Examples include the HGA of Vidal et al. (2012) and the EAX-based method of Nagata, Braysy, and Dullaert (2010).
- ② **Tabu Search + path relinking:** Rochat and Taillard (1995) maintain a pool of elite solutions. When the TS is diversified, it “path-relinks” the current solution toward an elite solution, exploring the corridor of solutions between the two.
- ③ **Scatter Search and GRASP:** Scatter search combines a reference set of diverse good solutions; GRASP (Greedy Randomised Adaptive Search Procedure) alternates between randomised constructive phases and local-search improvement phases.

Hybridizations III

Population-based Hybridization with Diversity Management

One of the key challenges in hybrid GA is premature convergence: if all individuals in the population become too similar, crossover stops producing novel offspring. The HGA of Vidal et al. (2012) addresses this through an explicit **diversity control** mechanism:

- The population is split into a *feasible* and an *infeasible* sub-population (infeasible solutions violate capacity constraints but may lead to good feasible solutions after repair).
- A **biased fitness** measure combines solution quality (objective value) and a **diversity contribution** (measured as average Hamming distance to the closest neighbours in the population).
- Individuals with low diversity contribution are periodically removed (*survivor selection*).

This mechanism ensures that the population remains diverse throughout the search, sustaining effective crossover and leading to high-quality solutions.

Decomposition and Parallelism

For very large instances (thousands of customers), parallelisation is essential. Several parallel strategies have been proposed:

- **Spatial decomposition:** Partition the service area into sub-regions (sectors, strips, or Voronoi cells) and solve each sub-region independently (Figure 1).
- **Independent runs with elite exchange:** Run P independent metaheuristic threads and periodically exchange elite solutions between threads.
- **Centralised neighbourhood evaluation:** Use parallel threads to evaluate the neighbourhood in parallel, reducing the time per iteration.

Hybridizations V

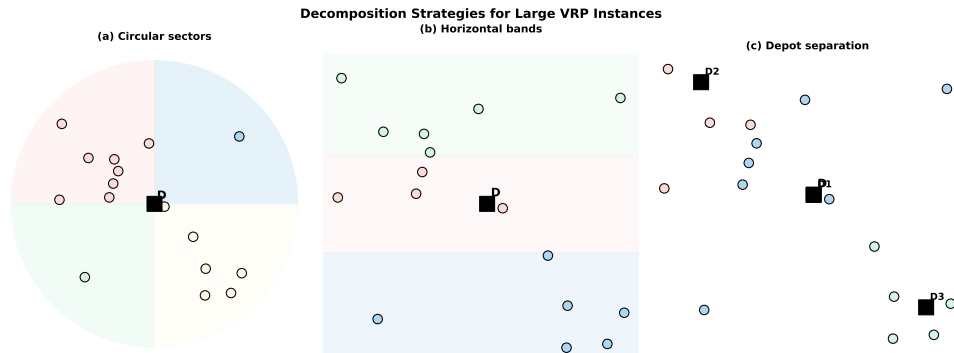


Figure: Three spatial decomposition strategies for large VRP instances: (a) circular sectors, (b) horizontal bands, (c) assignment to nearest depot. Each colour denotes a separate sub-problem solved independently.

Unified Algorithms for Multi-Attribute VRP I

The vast majority of real-world VRP instances involve additional constraints beyond simple capacity: time windows, heterogeneous fleets, multiple depots, split deliveries, backhauls, periodic scheduling, and so on. Historically, a separate algorithm was designed for each variant. **Unified algorithms** aim to solve a broad family of VRP variants with a *single* algorithmic framework, adapting to specific attributes through parameterised constraint handling.

Unified Algorithms for Multi-Attribute VRP II

The Unified HGA of Vidal et al. (2014)

Vidal, Crainic, Gendreau, and Prins proposed a unified hybrid GA that handles a large class of multi-attribute VRPs. The key design principles are:

- 1 **Giant-tour representation:** Solutions are encoded as a single sequence of all customers (a “giant tour”). The partition into routes is determined on the fly by an efficient *split procedure*.
- 2 **Split procedure:** Given a giant tour $(\pi_1, \pi_2, \dots, \pi_n)$, the split algorithm solves a shortest-path problem on an auxiliary graph to find the optimal partition respecting capacity and other constraints. This runs in $O(n)$ time for CVRP and $O(n^2)$ for VRPTW.
- 3 **Neighbourhood:** The GENIUS-style crossover and Or-opt / 2-opt / 3-opt local search operators are applied to the giant tour, with the split procedure converting the result back to routes at each evaluation.

Unified Algorithms for Multi-Attribute VRP III

Advantages of the Unified Approach

- A **single codebase** handles CVRP, VRPTW, multi-depot VRP, periodic VRP, heterogeneous fleet VRP, and combinations thereof.
- The **split procedure** automatically enforces all feasibility constraints without requiring ad-hoc repair operators for each variant.
- **State-of-the-art performance** on all tested variants: the unified HGA achieved competitive or record-breaking solutions on CVRP, VRPTW, and several other benchmark families.

The Adaptive Attribute Selection Paradigm

Vidal et al. (2014) extend the unified framework further by introducing **adaptive attribute selection**: the algorithm automatically identifies which problem attributes (time windows, depot assignment, etc.) are “active” for a given instance and adjusts the split procedure and neighbourhood operators accordingly. This makes the framework applicable to instances with up to 10 or more simultaneous attributes.

Key Insight

The giant-tour representation with split decoding elegantly separates the combinatorial structure of route construction from the constraint evaluation. It allows a single evolutionary operator (crossover) applied to the giant tour to implicitly explore an exponentially large space of feasible route partitions.

Benchmark Instances and Comparison Methodology I

The chapter presents a systematic computational comparison of metaheuristics on two standard benchmark sets.

Benchmark Data Sets

- ① **GWKC (Golden-Wasil-Kelly-Cheung)** instances: 20 large CVRP instances with $n \in \{240, 255, 280, 300, 320, 360, 396, 400, 420, 440, 480, 483\}$ customers and various numbers of depots (C = Christofides-type, CD = combined). These are the primary benchmark for comparing state-of-the-art metaheuristics.
- ② **CMT (Christofides-Mingozi-Toth)** instances: 14 classical instances with $n \in \{50, 75, 100, 150, 199\}$ customers. These smaller instances admit near-optimal solutions and serve as ground truth for exact methods.

Benchmark Instances and Comparison Methodology II

Performance Metrics

Two metrics are reported for each algorithm on each instance:

- **Solution value:** total travel distance of the best solution found.
- **Percentage gap:** $\text{gap} = \frac{c(s_{\text{alg}}) - c(s^*)}{c(s^*)} \times 100\%$, where $c(s^*)$ is the best known solution value across all algorithms and runs.

Running times are **normalised** to a reference machine speed using a benchmark program, making times comparable across different hardware. The normalised time unit corresponds roughly to a 1 GHz processor running a standard sorting benchmark.

Benchmark Instances and Comparison Methodology III

Algorithms in the Comparison

The following selected metaheuristics are compared (Table 4.2 of the book):

ID	Algorithm	Type
CLM01	Cordeau, Laporte, Mercier (2001)	Tabu Search
TV03	Tarantilis, Vortas (2003)	SA / TS hybrid
RDH04	Reimann, Doerner, Hartl (2004)	ACO
T05	Toth, Vigo — granular TS (2003/2005)	Tabu Search
MB07	Mester, Bräysy (2007)	SA hybrid
PRO7	Prins (2004, 2009)	GA
NB09	Nagata, Bräysy (2009)	EAX GA
P09	Pisinger, Ropke (2010)	ALNS
ZK10	Zachariadis, Kiranoudis (2010)	TS
GGW10	Groer, Golden, Wasil (2010)	CLAM
VCGLR12	Vidal et al. (2012)	HGA
GGW11	Groer, Golden, Wasil (2011)	—
CGW11	Chu, Gao, Wang (2011)	—
CM12	Contardo, Martinelli (2012)	TS
JCL12	Jin, Chen, Lu (2012)	EA

Computational Results: GWKC Instances I

Tables 4.3–4.9 in the book report detailed results for all 20 GWKC instances. We summarise the key findings here.

Selected Results from Table 4.7 (Part I)

#	n	Best known	CLM01 (%)	TV03 (%)	RDH04 (%)
1	240C	5623.47	5681.97 (1.04)	5736.15 (2.00)	5644.02 (0.37)
4	480C	13592.88	13740.60 (1.09)	14910.62 (9.69)	13699.11 (0.78)
5	200C	6460.98	6756.44 (4.57)	6697.53 (3.66)	6460.98 (0.00)
8	440C	11635.34	11869.50 (2.01)	12036.24 (3.45)	11936.16 (2.59)
11	399CD	912.84	929.07 (1.78)	936.04 (2.54)	927.27 (1.58)
Average gap			1.79%	2.06%	0.93%

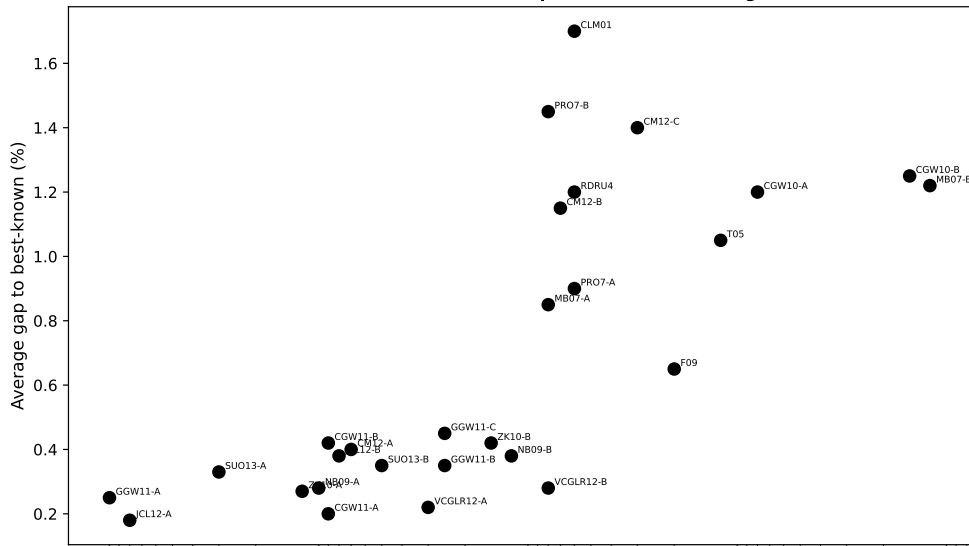
Computational Results: GWKC Instances II

Selected Results: Modern Methods (Tables 4.8–4.9)

ID	Avg. gap (%)	Norm. time (s)	Type
MB07 (best cfg)	0.33	13005.5	SA hybrid
NB09 (best cfg)	0.17	13005.5	EAX GA
P09 (best cfg)	0.63	233.4	ALNS
GGW10 (best cfg)	0.12	43.4	CLAM
VCGLR12 (best)	0.16	3387.5	HGA
GGW11 (best cfg)	0.19	43.4	—
CM12 (best cfg)	0.56	18770.0	TS
JCL12 (best cfg)	0.35	200978.4	EA
SUO13 (best cfg)	0.40	39382.3	HGA
GGW11-A	0.25	100000	—

Computational Results: GWKC Instances III

Solution quality vs. computation time
(GWKC benchmark instances, reproduction of book Fig. 4.3)



Other Measures of Quality and Robustness I

Beyond average gap, the chapter discusses several additional evaluation dimensions that are important for a complete picture of algorithm performance.

Simplicity

A heuristic should be as simple as possible in its implementation, especially if it is to be applied in different problem settings. Algorithms that can be tuned with only a small number of parameters are preferred over those requiring elaborate configuration. For example, the ALNS framework requires only the weight-update rule and a set of operators, while some Tabu Search implementations have dozens of interdependent parameters.

Robustness Across Instance Types

An algorithm is **robust** if it achieves consistently good performance across all instances in a benchmark set, not just on average. The standard deviation of the gap across instances is an important secondary metric. Some algorithms achieve a low average gap but have high variance (performing excellently on some instances but poorly on others).

Other Measures of Quality and Robustness II

Taking Advantage of Parallel Processing

Modern processors are massively parallel. The chapter highlights that algorithms which run k independent threads and report the best solution have an inherent advantage: running k independent copies of a randomised algorithm for time T/k each produces results that are statistically comparable to running a single copy for time T , but all k copies can run simultaneously on a multi-core machine.

Conclusions and Future Research Directions I

Chapter 4 has presented a comprehensive survey of heuristics for the Vehicle Routing Problem, from the classical constructive methods of the 1960s–1970s to the sophisticated hybrid metaheuristics of the 2010s. Several clear conclusions emerge from the computational evidence.

Main Conclusions

- ① **Metaheuristics dominate:** For any VRP instance large enough that exact methods are impractical, a well-implemented metaheuristic (Tabu Search, ALNS, or HGA) will find a solution within 0.5% of the best known value.
- ② **Hybridization pays off:** Combining local search with population-based search (memetic algorithms) consistently outperforms either component in isolation.
- ③ **Unified frameworks are the future:** As real-world VRP instances involve increasingly many simultaneous constraints, algorithms that can handle multiple attributes without redesign are highly valuable.
- ④ **Benchmarks are essential:** The availability of standardised benchmark sets (GWKC, CMT, Solomon) and a common normalised time scale has enabled meaningful comparison across algorithms and hardware generations.

Conclusions and Future Research Directions III

Future Research Directions

The chapter identifies several promising directions for future work:

- **Robustness vs. time:** Current top algorithms are often sensitive to parameter settings. Developing parameter-free or self-calibrating methods would make them more accessible.
- **Very large-scale instances:** Instances with tens of thousands of customers are increasingly common in e-commerce logistics. Decomposition methods and massively parallel algorithms are needed.
- **Dynamic and stochastic VRP:** Real-world routing involves uncertainty (stochastic demands, traffic disruptions, new customer requests arriving in real time). Metaheuristics need to be adapted to re-optimize solutions online with very short re-computation windows.
- **Machine learning integration:** Recent research explores using learned heuristics (neural combinatorial optimisation) either as standalone solvers or as components within classical metaheuristic frameworks (e.g., to learn good destroy/repair operators for LNS).
- **Multi-objective VRP:** Optimising simultaneously for cost, CO₂ emissions, and driver workload requires Pareto-front search methods adapted from multi-objective evolutionary computing.

Algorithm Summary and Complexity Reference I

The following table summarises the key heuristics covered in this chapter, their type, neighbourhood complexity, and typical solution quality on standard CVRP benchmarks.

Algorithm	Type	Neighbourhood	Complexity	Typical gap
Clarke-Wright	Constructive	—	$O(n^2 \log n)$	10–15%
Nearest Neighbour	Constructive	—	$O(n^2)$	20–25%
Sweep	Constructive	—	$O(n \log n)$	10–20%
2-opt	Local search	$O(n^2)$ per route	$O(n^2)$	5–10%
3-opt	Local search	$O(n^3)$ per route	$O(n^3)$	3–7%
Or-opt-1	Local search	$O(n^2)$	$O(n^2)$	5–8%
Lin-Kernighan	Local search	Variable	$O(n^{2.5})$	1–3%
Tabu Search	Metaheuristic	$O(n^2)$ per iter	—	0.5–2%
Sim. Annealing	Metaheuristic	$O(n)$ per iter	—	0.5–2%
Genetic Alg.	Metaheuristic	Population-based	—	0.3–1%
ACO	Metaheuristic	Population-based	—	0.5–1%
LNS / ALNS	Metaheuristic	$O(\binom{n}{q})$	—	0.2–0.5%
Hybrid GA (HGA)	Hybrid	Population+LS	—	0.1–0.3%

Algorithm Summary and Complexity Reference II

Note: “typical gap” refers to the gap from the best known solution on CVRP benchmark instances (GWKC or CMT), and reflects the algorithm applied in isolation with standard parameter settings. Hybrid implementations and careful parameter tuning generally achieve lower gaps than those shown here.

Key References I

- [1] G. Clarke and J. W. Wright, *Scheduling of vehicles from a central depot to a number of delivery points*, Operations Research, 12 (1964), pp. 568–581.
- [2] B. E. Gillett and L. R. Miller, *A heuristic algorithm for the vehicle-dispatch problem*, Operations Research, 22 (1974), pp. 340–349.
- [3] S. Lin, *Computer solutions of the traveling salesman problem*, Bell System Technical Journal, 44 (1965), pp. 2245–2269.
- [4] S. Lin and B. W. Kernighan, *An effective heuristic algorithm for the traveling-salesman problem*, Operations Research, 21 (1973), pp. 498–516.
- [5] I. Or, *Traveling salesman-type combinatorial problems and their relation to the logistics of regional blood banking*, Ph.D. thesis, Northwestern University, 1976.
- [6] F. Glover, *Future paths for integer programming and links to artificial intelligence*, Computers & Operations Research, 13 (1986), pp. 533–549.

Key References II

- [7] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, *Optimization by simulated annealing*, Science, 220 (1983), pp. 671–680.
- [8] J. H. Holland, *Adaptation in Natural and Artificial Systems*, University of Michigan Press, Ann Arbor, MI, 1975.
- [9] M. Dorigo, V. Maniezzo, and A. Coloni, *Ant system: Optimization by a colony of cooperating agents*, IEEE Transactions on Systems, Man, and Cybernetics B, 26 (1996), pp. 29–41.
- [10] P. Shaw, *Using constraint programming and local search methods to solve vehicle routing problems*, in Proceedings of CP-98, 1998, pp. 417–431.
- [11] D. Pisinger and S. Ropke, *A general heuristic for vehicle routing problems*, Computers & Operations Research, 34 (2007), pp. 2403–2435.
- [12] T. Vidal, T. G. Crainic, M. Gendreau, N. Lahrichi, and W. Rei, *A hybrid genetic algorithm for multidepot and periodic vehicle routing problems*, Operations Research, 60 (2012), pp. 611–624.

Key References III

- [13] T. Vidal, T. G. Crainic, M. Gendreau, and C. Prins, *A unified solution framework for multi-attribute vehicle routing problems*, European Journal of Operational Research, 234 (2014), pp. 658–673.
- [14] Y. Nagata and O. Bräysy, *A penalty-based edge assembly memetic algorithm for the vehicle routing problem with time windows*, Computers & Operations Research, 36 (2009), pp. 2026–2043.
- [15] P. Toth and D. Vigo, *The granular tabu search and its application to the vehicle-routing problem*, INFORMS Journal on Computing, 15 (2003), pp. 333–346.
- [16] J.-F. Cordeau, G. Laporte, and A. Mercier, *A unified Tabu Search heuristic for vehicle routing problems with time windows*, Journal of the Operational Research Society, 52 (2001), pp. 928–936.
- [17] C. D. Tarantilis, C. T. Kiranoudis, and V. S. Vassiliadis, *A list-based threshold accepting algorithm for the capacitated vehicle routing problem*, International Journal of Computer Mathematics, 79 (2002), pp. 537–553.
- [18] O. Bräysy, W. Dullaert, and M. Gendreau, *Evolutionary algorithms for the vehicle routing problem with time windows*, Journal of Heuristics, 10 (2004), pp. 587–611.